Title: Implementation of a Simple Storage Smart Contract using Remix IDE

code:

```
from web3 import Web3

import tkinter as tk

from tkinter import messagebox

import time

from collections import defaultdict

import os


# Step 1: Connect to the blockchain network

# Using Ethereum Sepolia testnet with a public RPC endpoint

sepolia_url = 'https://ethereum-sepolia-rpc.publicnode.com'

web3 = Web3(Web3.HTTPProvider(sepolia_url))


class SimpleStorageSimulator:

    """Simulate a blockchain smart contract with state management
and transactions."""


    def __init__(self):

        self.stored_value = 0  # State variable equivalent to uint256
private storedValue

        self.transactions = []  # Log transactions with timestamps


    def set(self, value):
```

```
        """Simulate set function: store value (transaction-like, logs
timestamp)"""
        if not isinstance(value, int):
            raise TypeError("Value must be an integer.")
        if value < 0:
            raise ValueError("Value must be non-negative.")

        timestamp = int(time.time())
        self.stored_value = value

        # Generate simulated hash and block number
        import hashlib
        tx_hash =
hashlib.sha256(f"{len(self.transactions)}{value}{timestamp}".encode()
).hexdigest()[:16]
        block_number = 18000000 + len(self.transactions)

        self.transactions.append({
            'type': 'set',
            'value': value,
            'timestamp': timestamp,
            'hash': f"0x{tx_hash}",
            'block': block_number
        })
        return f"✓ Transaction: set({value}) at {time.ctime(timestamp)}"
```

```
    def get(self):
        """Simulate get function: retrieve value (view, no state
change)"""
        return self.stored_value

    def generate_transaction_report(self):
        """Generate a simple report of all transactions."""
        report = "\n" + "=" * 50
        report += "\nTransaction Report:\n"
        report += "=" * 50 + "\n"
        for i, tx in enumerate(self.transactions, 1):
            tx_time = time.ctime(tx['timestamp'])
            report += f"{i}. Type: {tx['type']}, Value: {tx['value']},
Timestamp: {tx_time}\n"
        report += "=" * 50 + "\n"
        return report

    def generate_html_report(self,
filename="smart_contract_report.html"):
        """Generate an HTML report of the contract and transactions."""
        # Generate transaction rows
        transaction_rows = ""
        for i, tx in enumerate(self.transactions, 1):
            tx_time = time.ctime(tx['timestamp'])
```

```
        tx_hash = tx.get('hash', 'N/A')
        tx_block = tx.get('block', 'N/A')
        transaction_rows +=
f"<tr><td>{i}</td><td>{tx['type']}</td><td>{tx['value']}</td><td>{tx_
block}</td><td>{tx_hash}</td><td>{tx_time}</td><td><span
class='success'>SUCCESS</span></td></tr>"


    html_content = f"""<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>SimpleStorage Smart Contract Report</title>
  <style>
    body {{ font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-
serif; background: linear-gradient(135deg, #667eea 0%, #764ba2
100%); min-height: 100vh; padding: 20px; }}
    .container {{ max-width: 1200px; margin: 0 auto; background:
white; border-radius: 10px; box-shadow: 0 10px 40px rgba(0,0,0,0.3);
overflow: hidden; }}
    .header {{ background: linear-gradient(135deg, #667eea 0%,
#764ba2 100%); color: white; padding: 30px; text-align: center; }}
    .header h1 {{ font-size: 2.5em; margin-bottom: 10px; }}
    .content {{ padding: 30px; }}
    .section {{ margin-bottom: 30px; }}
```

```
        .section h2 {{ color: #667eea; border-bottom: 2px solid #667eea;
padding-bottom: 10px; margin-bottom: 15px; }}
        .stats {{ display: grid; grid-template-columns: repeat(auto-fit,
minmax(200px, 1fr)); gap: 15px; margin-bottom: 20px; }}
        .stat-box {{ background: #f0f0f0; padding: 20px; border-radius:
8px; border-left: 4px solid #667eea; }}
        .stat-label {{ font-size: 0.9em; color: #666; margin-bottom: 5px; }}
        .stat-value {{ font-size: 1.8em; color: #667eea; font-weight: bold;
word-break: break-all; }}
        table {{ width: 100%; border-collapse: collapse; margin-top:
15px; font-size: 0.9em; }}
        th {{ background: #667eea; color: white; padding: 12px; text-
align: left; }}
        td {{ padding: 10px; border-bottom: 1px solid #ddd; word-break:
break-all; }}
        tr:hover {{ background: #f9f9f9; }}
        .success {{ color: #27ae60; font-weight: bold; }}
        .timestamp {{ color: #999; font-size: 0.9em; }}
        .footer {{ background: #f0f0f0; padding: 20px; text-align: center;
color: #666; }}
    </style>
</head>
<body>
    <div class="container">
        <div class="header">
            <h1>Blockchain Smart Contract Report</h1>
```

```
        <p>Transaction Details with Hash and Block Information</p>
    </div>
    <div class="content">
        <div class="section">
            <h2>Contract Statistics</h2>
            <div class="stats">
                <div class="stat-box">
                    <div class="stat-label">Current Value</div>
                    <div class="stat-value">{self.stored_value}</div>
                </div>
                <div class="stat-box">
                    <div class="stat-label">Total Transactions</div>
                    <div class="stat-value">{len(self.transactions)}</div>
                </div>
            </div>
        </div>
        <div class="section">
            <h2>Transaction History</h2>
            <table>
                <thead>
                    <tr><th>#</th><th>Function</th><th>Value</th><th>B
lock</th><th>Hash</th><th>Timestamp</th><th>Status</th></tr>
                </thead>
                <tbody>
```

```
                {transaction_rows}
            </tbody>
        </table>
    </div>
    <div class="section">
        <h2>Contract Information</h2>
        <p><strong>Contract Name:</strong> SimpleStorage</p>
        <p><strong>State Variable:</strong> storedValue
(uint256)</p>
        <p><strong>Functions:</strong> set(uint256), get()</p>
    </div>
  </div>
  <div class="footer">
    <p>Generated on: <span
class="timestamp">{time.strftime("%Y-%m-%d
%H:%M:%S")}</span></p>
  </div>
 </div>
</body>
</html>"""

  # Write to file
  with open(filename, 'w') as f:
    f.write(html_content)
```

```
        return filename


    def generate_tinkercad_display(self,
filename="tinkercad_led_display.html"):
        """Generate an HTML file to simulate the Tinkercad LED
display."""
        # Map value to LED grid position (5x5 grid = 25 positions)
        val = self.stored_value % 26
        grid = [['.' for _ in range(5)] for _ in range(5)]
        row, col = divmod(val, 5)
        grid[row][col] = 'X'

        html_content = """
        <!DOCTYPE html>
        <html lang="en">
        <head>
            <meta charset="UTF-8">
            <meta name="viewport" content="width=device-width, initial-
scale=1.0">
            <title>Tinkercad LED Display</title>
            <style>
                body {
                    font-family: Arial, sans-serif;
                    text-align: center;
                    background-color: #f4f4f4;
```

```
    margin: 0;
    padding: 20px;
}
.container {
    max-width: 600px;
    margin: 0 auto;
    background: white;
    padding: 30px;
    border-radius: 10px;
    box-shadow: 0 5px 20px rgba(0,0,0,0.1);
}
h1 {
    color: #667eea;
    margin-bottom: 20px;
}
.info {
    background: #f0f0f0;
    padding: 15px;
    border-radius: 5px;
    margin-bottom: 20px;
}
.grid {
    display: grid;
    grid-template-columns: repeat(5, 60px);
```

```
            grid-gap: 8px;
            justify-content: center;
            margin: 30px auto;
        }
        .cell {
            width: 60px;
            height: 60px;
            background-color: #ddd;
            border: 2px solid #ccc;
            border-radius: 8px;
            display: flex;
            align-items: center;
            justify-content: center;
            font-weight: bold;
            color: #999;
        }
        .cell.on {
            background-color: #ff5722;
            border-color: #ff5722;
            color: white;
            box-shadow: 0 0 10px rgba(255, 87, 34, 0.6);
        }
    </style>
</head>
```

```
        <body>
            <div class="container">
                <h1>Tinkercad LED Display</h1>
                <div class="info">
                    <p><strong>Current Value:</strong> """ +
str(self.stored_value) + """</p>
                    <p><strong>LED Position:</strong> Row """ + str(row) +
""", Column """ + str(col) + """</p>
                </div>
                <div class="grid">
        """

        cell_index = 0
        for row_idx, row_data in enumerate(grid):
            for col_idx, cell in enumerate(row_data):
                if cell == 'X':
                    html_content += f'<div class="cell on">{cell_index}</div>'
                else:
                    html_content += f'<div class="cell">.</div>'
                cell_index += 1

        html_content += """
                </div>
            </div>
```

```
        </body>
        </html>
        """

        with open(filename, "w") as f:
            f.write(html_content)

        return filename

class BlockchainGUI:
    """Tkinter GUI for blockchain smart contract simulator and web3 integration."""

    def __init__(self, root):
        self.root = root
        self.root.title("Blockchain Smart Contract Simulator")
        self.root.geometry("700x600")
        self.root.configure(bg="#f0f0f0")

        # Initialize contract
        self.contract = SimpleStorageSimulator()

        # Check blockchain connection
        self.is_connected = False
```

```
        self.check_blockchain_connection()

        # Create UI
        self.create_ui()

    def check_blockchain_connection(self):
        """Check if connected to the blockchain network."""
        try:
            if web3.is_connected():
                self.is_connected = True
                self.chain_id = web3.eth.chain_id
                self.block_number = web3.eth.block_number
            else:
                self.is_connected = False
        except Exception as e:
            self.is_connected = False
            print(f"Connection error: {e}")

    def create_ui(self):
        """Create the GUI elements."""
        # Title
        title = tk.Label(self.root, text="🔗 Blockchain Smart Contract Simulator",
```

```
                font=("Arial", 16, "bold"), bg="#667eea", fg="white",
pady=10)
    title.pack(fill=tk.X)

    # Blockchain Status
    status_frame = tk.Frame(self.root, bg="white", relief=tk.RAISED,
bd=1)
    status_frame.pack(fill=tk.X, padx=10, pady=10)

    status_text = "✓ Connected to Ethereum Sepolia" if
self.is_connected else "✗ Disconnected from Blockchain"
    status_color = "green" if self.is_connected else "red"

    tk.Label(status_frame, text="Blockchain Status:", font=("Arial",
10, "bold"), bg="white").pack(anchor=tk.W, padx=10, pady=5)
    tk.Label(status_frame, text=status_text, font=("Arial", 10),
fg=status_color, bg="white").pack(anchor=tk.W, padx=20)

    if self.is_connected:
        tk.Label(status_frame, text=f"Chain ID: {self.chain_id}",
                 font=("Arial", 9), bg="white",
fg="gray").pack(anchor=tk.W, padx=20)
        tk.Label(status_frame, text=f"Latest Block Number:
{self.block_number}",
                 font=("Arial", 9), bg="white",
fg="gray").pack(anchor=tk.W, padx=20, pady=5)
```

```
        # Contract Section
        contract_frame = tk.LabelFrame(self.root, text="Smart Contract
Operations", font=("Arial", 12, "bold"),
                                bg="white", padx=10, pady=10)
        contract_frame.pack(fill=tk.BOTH, expand=True, padx=10,
pady=10)

        # Current Value Display
        tk.Label(contract_frame, text="Current Value:", font=("Arial",
10), bg="white").pack(anchor=tk.W, pady=5)
        self.value_label = tk.Label(contract_frame,
text=str(self.contract.get()), font=("Arial", 14, "bold"),
                                fg="#667eea", bg="white")
        self.value_label.pack(anchor=tk.W, padx=20, pady=5)

        # Input Section
        input_frame = tk.Frame(contract_frame, bg="white")
        input_frame.pack(fill=tk.X, pady=10)

        tk.Label(input_frame, text="Enter Value:", font=("Arial", 10),
bg="white").pack(side=tk.LEFT, padx=5)
        self.value_entry = tk.Entry(input_frame, font=("Arial", 10),
width=15)
        self.value_entry.pack(side=tk.LEFT, padx=5)
```

```
        tk.Button(input_frame, text="Set Value",
command=self.set_value,
                  bg="#667eea", fg="white", font=("Arial", 10),
padx=10).pack(side=tk.LEFT, padx=5)

        # Transactions Display
        tk.Label(contract_frame, text="Transactions:", font=("Arial", 10,
"bold"), bg="white").pack(anchor=tk.W, pady=(10, 5))

        self.transaction_text = tk.Text(contract_frame, height=8,
width=60, font=("Arial", 9),
                                        bg="#f9f9f9", relief=tk.SUNKEN, bd=1)
        self.transaction_text.pack(fill=tk.BOTH, expand=True, pady=5)

        # Button Frame
        button_frame = tk.Frame(self.root, bg="#f0f0f0")
        button_frame.pack(fill=tk.X, padx=10, pady=10)

        tk.Button(button_frame, text="Generate HTML Report",
command=self.generate_report,
                  bg="#27ae60", fg="white", font=("Arial", 10), padx=10,
pady=5).pack(side=tk.LEFT, padx=5)

        tk.Button(button_frame, text="Generate LED Display",
command=self.generate_display,
```

```
            bg="#2980b9", fg="white", font=("Arial", 10), padx=10,
pady=5).pack(side=tk.LEFT, padx=5)

        tk.Button(button_frame, text="Exit", command=self.root.quit,
            bg="#e74c3c", fg="white", font=("Arial", 10), padx=10,
pady=5).pack(side=tk.RIGHT, padx=5)

        # Update transactions display
        self.update_transactions()

    def set_value(self):
        """Set a new value in the contract."""
        try:
            value = int(self.value_entry.get())
            result = self.contract.set(value)
            self.value_label.config(text=str(self.contract.get()))
            self.value_entry.delete(0, tk.END)
            self.update_transactions()
            messagebox.showinfo("Success", result)
        except ValueError:
            messagebox.showerror("Error", "Please enter a valid integer.")
        except Exception as e:
            messagebox.showerror("Error", str(e))
```

```
    def update_transactions(self):
        """Update the transactions display."""
        self.transaction_text.config(state=tk.NORMAL)
        self.transaction_text.delete(1.0, tk.END)

        if self.contract.transactions:
            for i, tx in enumerate(self.contract.transactions, 1):
                tx_time = time.ctime(tx['timestamp'])
                tx_hash = tx.get('hash', 'N/A')
                tx_block = tx.get('block', 'N/A')
                self.transaction_text.insert(tk.END, f"{i}. Value: {tx['value']},
Block: {tx_block}, Hash: {tx_hash}, Time: {tx_time}\n")
        else:
            self.transaction_text.insert(tk.END, "No transactions yet.")

        self.transaction_text.config(state=tk.DISABLED)

    def generate_report(self):
        """Generate HTML report."""
        try:
            filename = self.contract.generate_html_report()
            messagebox.showinfo("Success", f"Report generated:
{filename}")
            os.startfile(filename)
```

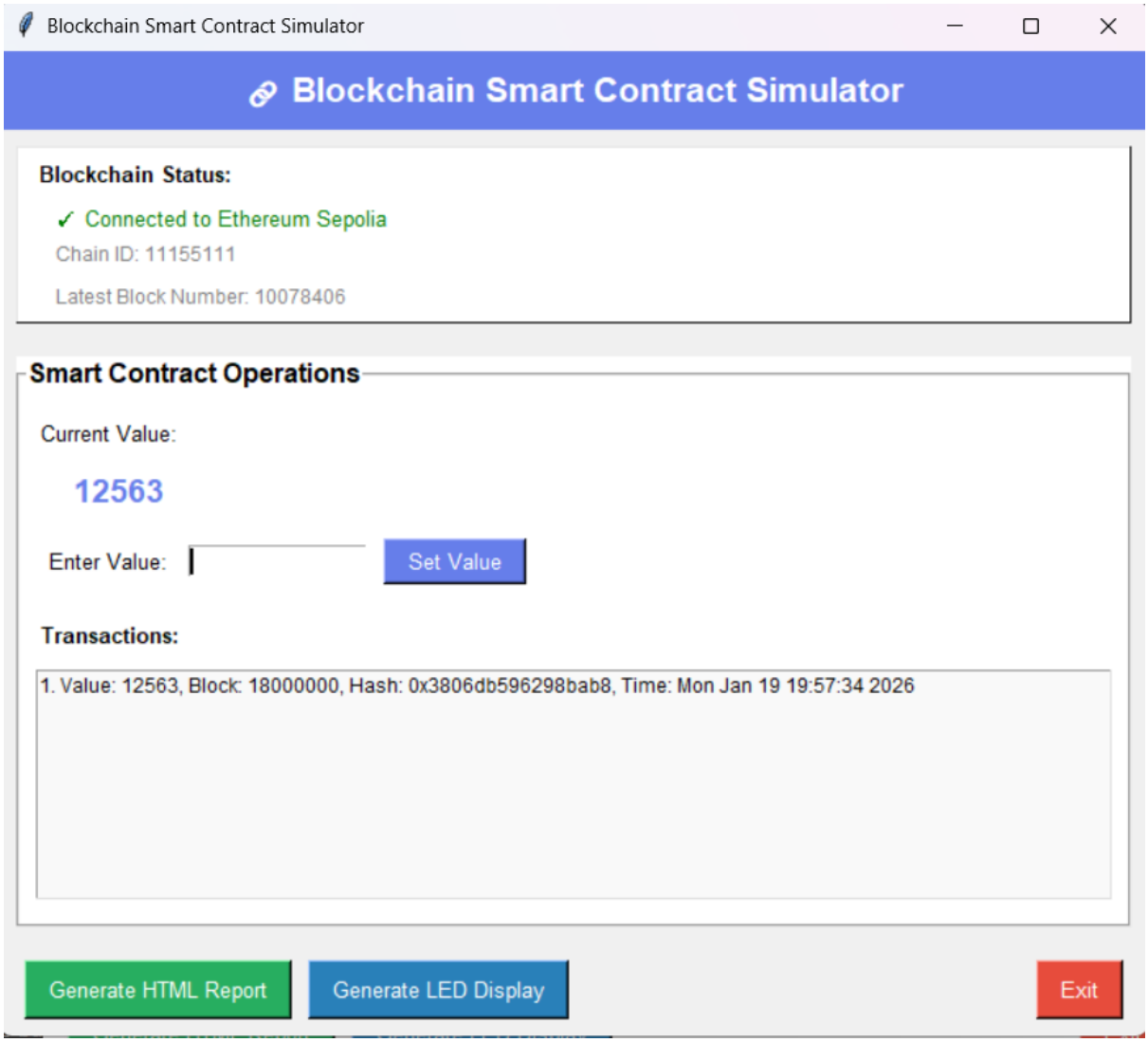
```
        except Exception as e:
            messagebox.showerror("Error", str(e))

    def generate_display(self):
        """Generate LED display."""
        try:
            filename = self.contract.generate_tinkercad_display()
            messagebox.showinfo("Success", f"LED display generated:
{filename}")
            os.startfile(filename)
        except Exception as e:
            messagebox.showerror("Error", str(e))

def main():
    """Main function to run the GUI."""
    root = tk.Tk()
    gui = BlockchainGUI(root)
    root.mainloop()

if __name__ == "__main__":
    main()
```

Output:


Blockchain Smart Contract Simulator
Blockchain Smart Contract Simulator
Blockchain Status:
✓ Connected to Ethereum Sepolia
Chain ID: 11155111
Latest Block Number: 10078406
Smart Contract Operations
Current Value:
12563
Enter Value:
Set Value
Transactions:
1. Value: 12563, Block: 18000000, Hash: 0x3806db596298bab8, Time: Mon Jan 19 19:57:34 2026
Generate HTML Report
Generate LED Display
Exit